

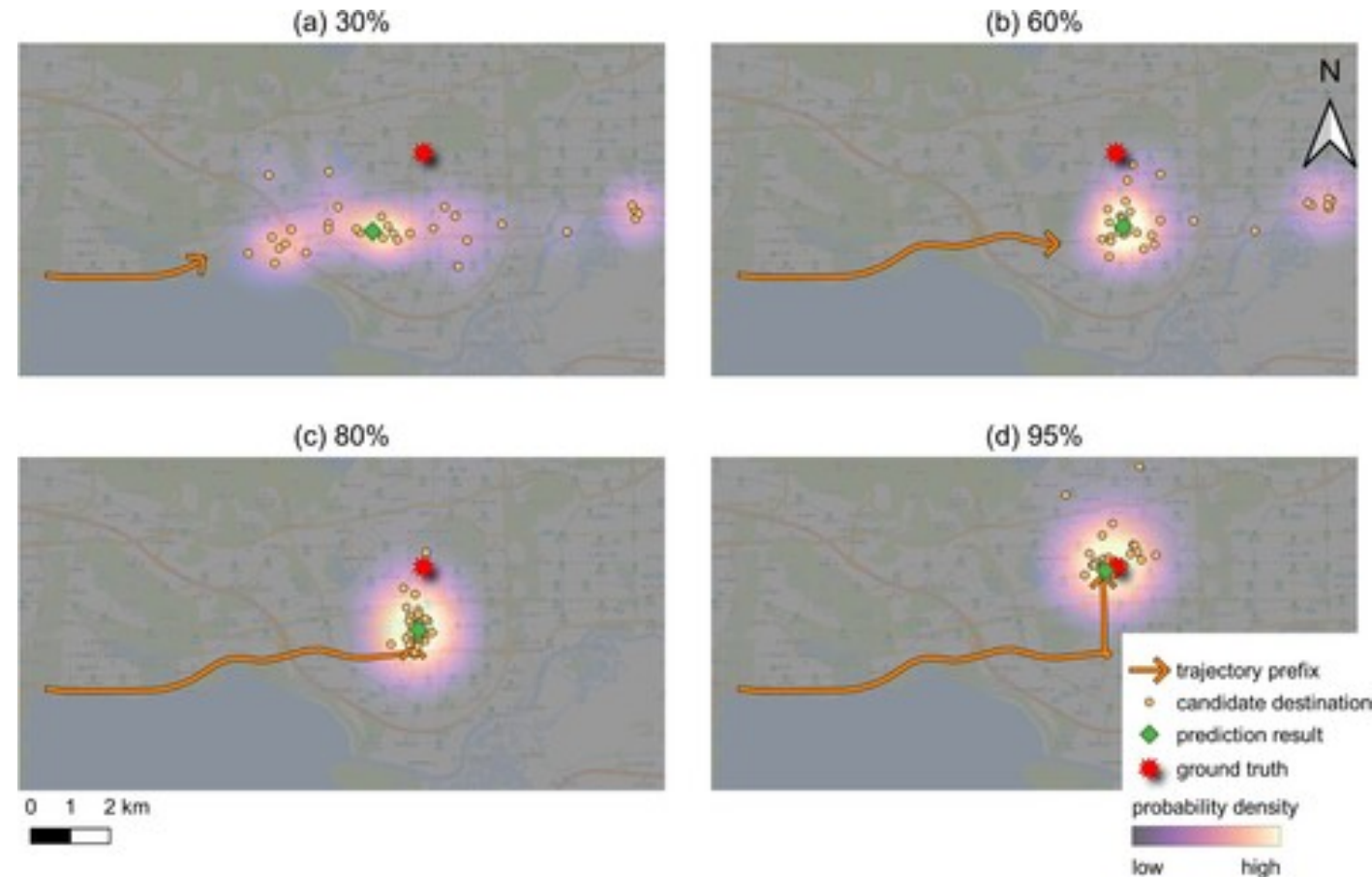
Conformal Prediction for Uncertainty-Aware Planning with Diffusion Dynamics Model

Jiankai Sun et al.

Chia Chien, Liu 4/17/2025

Background

- **Problem**
- Uncertainty in robotic environments
- Diffusion → not quantify uncertainty
- Offline RL → OOD risk,
no uncertainty bound
 - learns only from a fixed historical dataset,
without interacting with the environment
 - unseen states → predictions with **no uncertainty bound**
- **Goal: Reduce uncertainty, without accuracy loss**
- **PlanCP = diffusion + CP as training loss**



- As trajectory context increases, uncertainty decreases, and predictions become more concentrated and accurate.

Related work

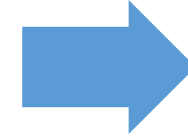
Diffusion Planning

- trajectory generation
- not quantify uncertainty



Conformal Prediction

- quantified uncertainty
- not train to reduce uncertainty



PlanCP

- trajectory planner
- quantified uncertainty
- train to reduce uncertainty

Diffusion Planning

Conformal Prediction

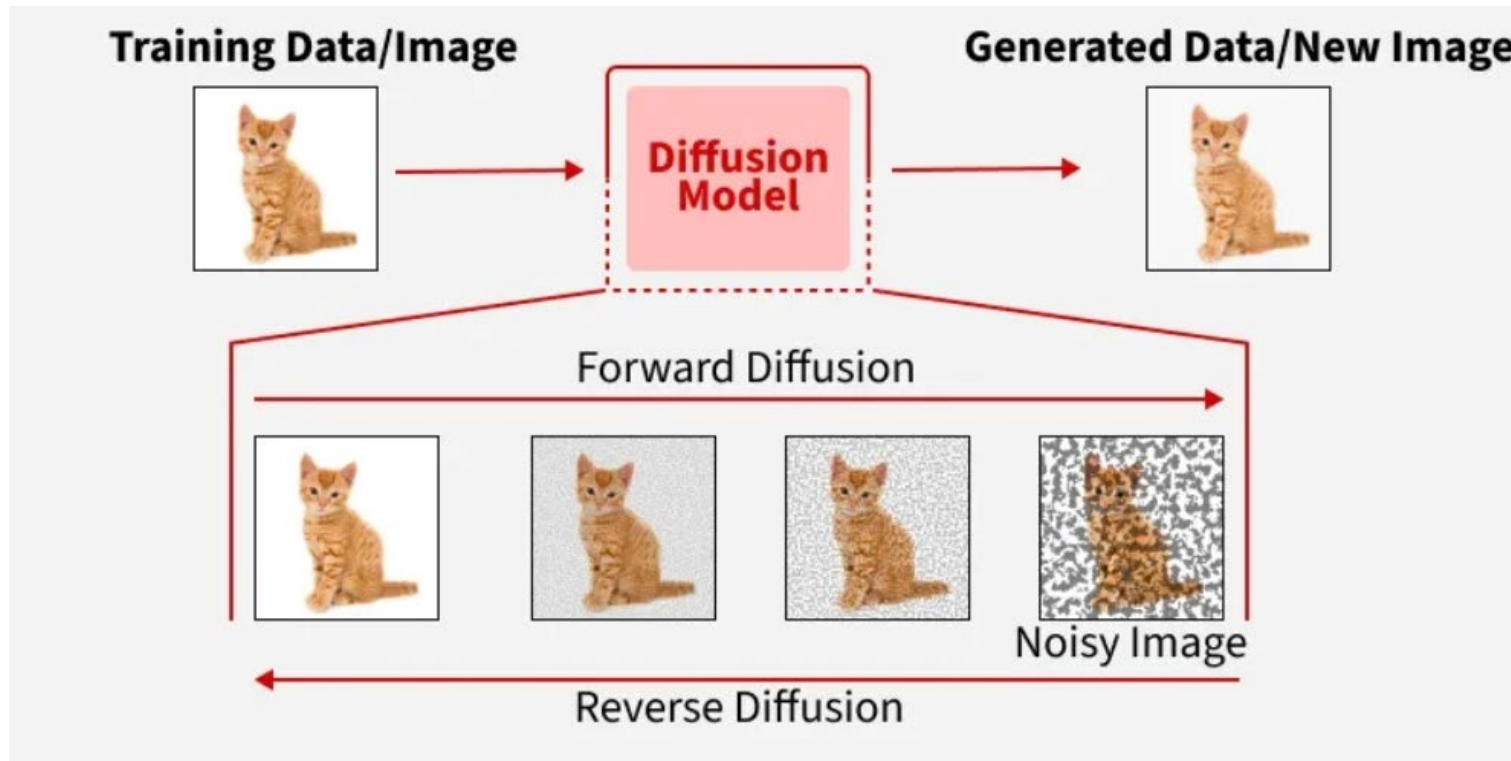
Diffuser

- $p_{\theta}(\tau^0) = \int p(\tau^N) \cdot \prod p_{\theta}(\tau^{n-1} | \tau^n) d\tau^{1:N}$
- $p_{\theta}(\tau^{n-1} | \tau^n) = \mathcal{N}(\tau^{n-1}; \mu_{\theta}(\tau^n, n), \Sigma_n)$
- diffusion model learns a trajectory distribution $p_{\theta}(\tau)$ through iterative denoising
- **not quantify, no bound on uncertainty**



planCP

- $P_{\theta}(\tau) \propto p_{\theta}(\tau) \cdot w(\tau)$
- $w(\tau)$ from reward model's by transformer attention
- generated $\tau_i \rightarrow \tilde{r}_i = \|\tau_i - \tau^E\|^2$ **nonconformity score**
- later derive to penalize for generating trajectories with high uncertainty



- What it do?
 - Calculating the error on calibration set
 - at least a $1 - \alpha$ prob that true answer will fall within the predicted interval.
- basic assumption
 - cal samples **exchangeable**
 - **Random fixed data offline <-> depend data online**

Advantages	
no internal model assumption	• cal from error, take quantile. • CP = "how different from other samples"
no data distribution assumption: finite samples guarantee	• As long as $n+1$ exchangeable samples → $\text{Prob}(\text{any rank}) = 1/(n+1)$
Customize confidence level $1-\alpha$	• Quantile $\hat{C} = \lceil (n+1)(1-\alpha) \rceil / n$

Prior

PlanCP

- predicted vs. true reward
- CP to quantify uncertainty,
cal to "provide coverage guarantees"
- not train to reduce uncertainty

- predicted vs. expert trajectory
- + train to reduce uncertainty

-

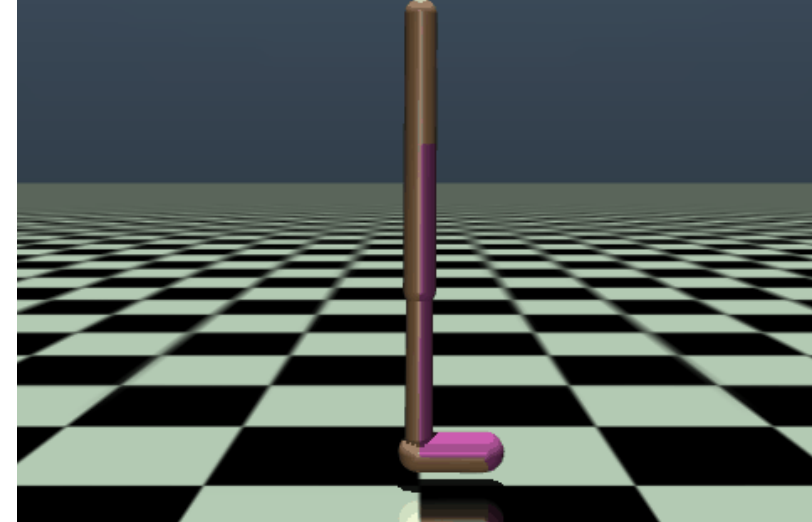
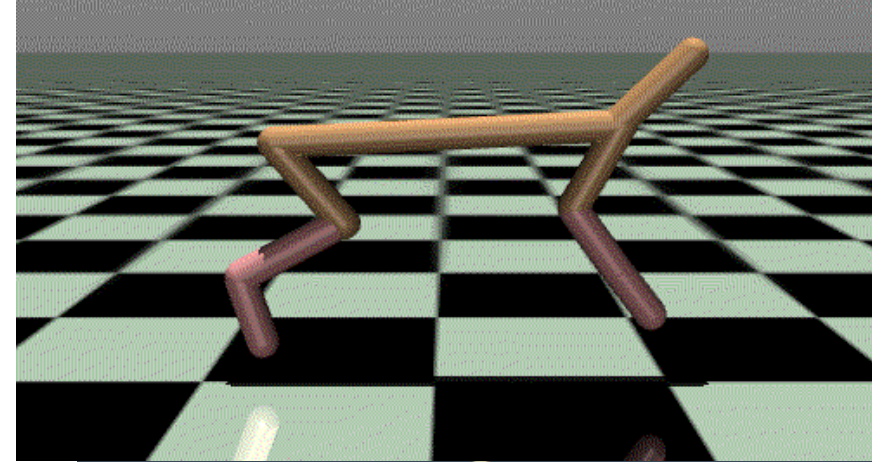
- $\tilde{r}_i = \|\tau_i - \tau_i^E\|^2$ nonconformity score
= distance from expert

- uncertainty bound

- $\hat{C} = \text{Quantile}(d_0, \dots, d_{i-1}, d_{i+1}, \dots, d_k ; [(k+1)(1-\alpha)] / k)$
- if $\tilde{r}_i > \hat{C}$, far from expert
→ penalize during training

Datasets from D4RL (Simulation environment)

- Maze2D (U-Maze, Medium, Large)
 - Sparse reward
 - test long-horizon planning
- Locomotion(HalfCheetah, Walker2d)
 - Quality: Medium-Expert, Medium, Medium-Replay
 - test offline RL
 - HalfCheetah (simple, run fast)
 - Walker2d (hard, maintain balance)



Experimental setups

- **Data Processing**

- D_train: 200,000 iterations → learns diffusion planner, reward model
- Split from eval data
 - D_cal (20%) builds reward interval $\hat{q}$
 - D_test (80%) verifies coverage guarantee

- **hyperparameter**

	Value	Role
lr	2×10^{-4}	Adam optimizer, gradient step size, standard default
λ	5	weight uncertainty loss (to total training loss)
α	0.1	failure prob

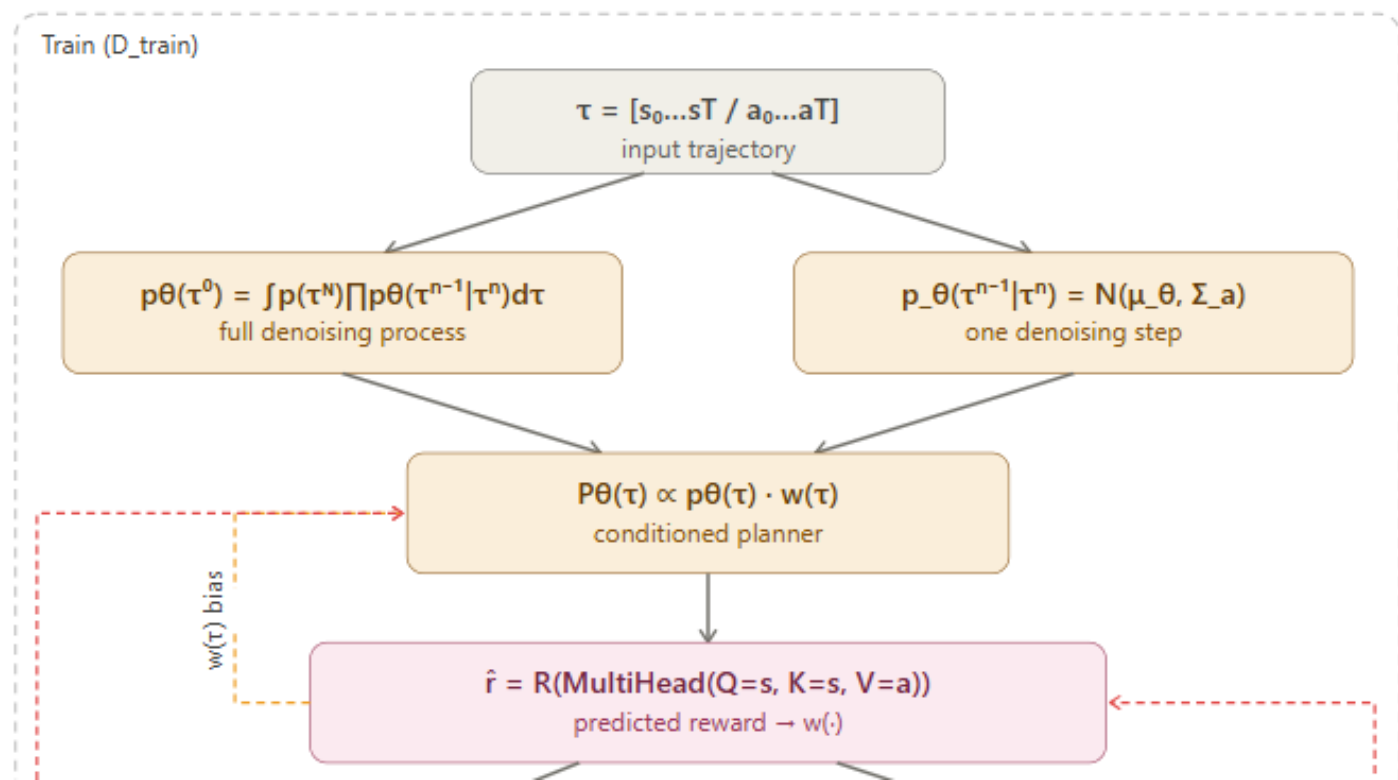
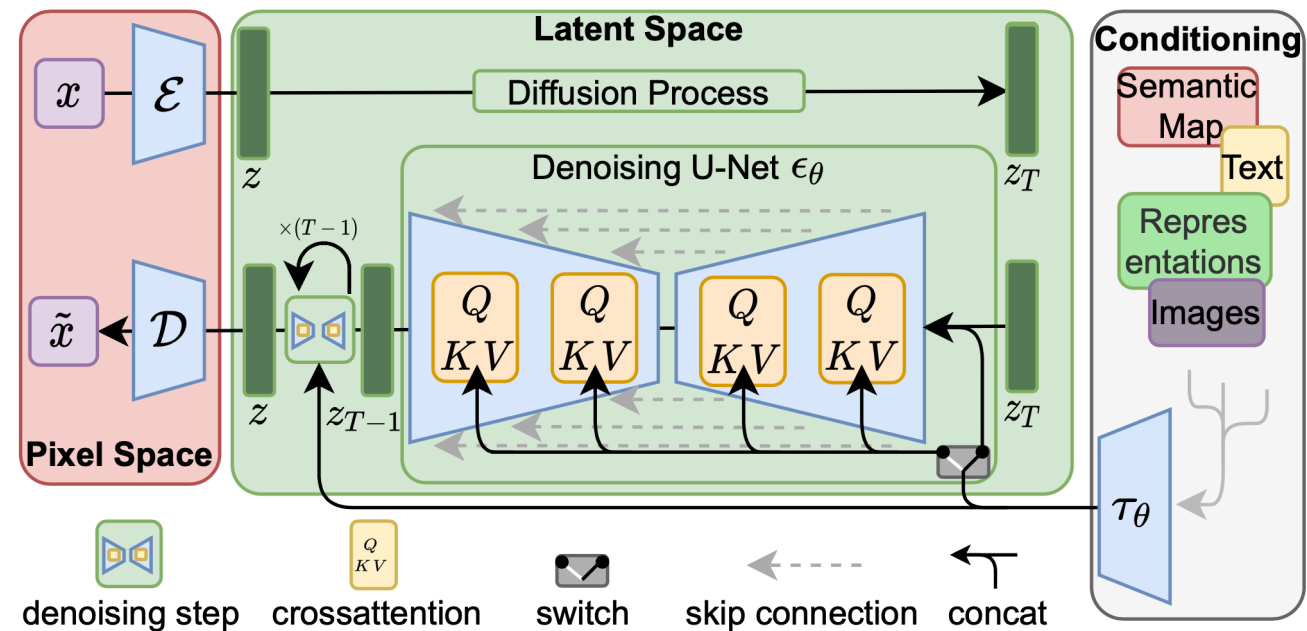
method – 1^o model architecture

• Diffusion Planner

- generate trajectory via denoising
- $P_{\theta}(\tau) \propto p_{\theta}(\tau) \cdot w(\tau)$
- Reward-weighted sampling toward high-reward trajectories

• Reward Model

- transformer attention over full trajectory
- $\hat{r}_{0:T} = R(\text{MultiHead}(Q=s_{0:T}, K=s_{0:T}, V=a_{0:T}))$
= reward prediction
- Captures long-range dependencies
- $\hat{r} \rightarrow w(\tau)$ to bias sampling toward high-reward trajectories



method – 2^o training

Conformal Prediction

- $\tilde{r}_i = \|\tau_i - \tau_i^E\|^2$
- Nonconformity score (trajectory error)
- $\hat{C} = \text{Quantile}(\tilde{r}, 1-\alpha) = L(\theta)_{\text{uncertainty}}$
- uncertainty upper bound
- **Used as constraint in diffusion planner training**
- **differentiable quantile → enable backprop**

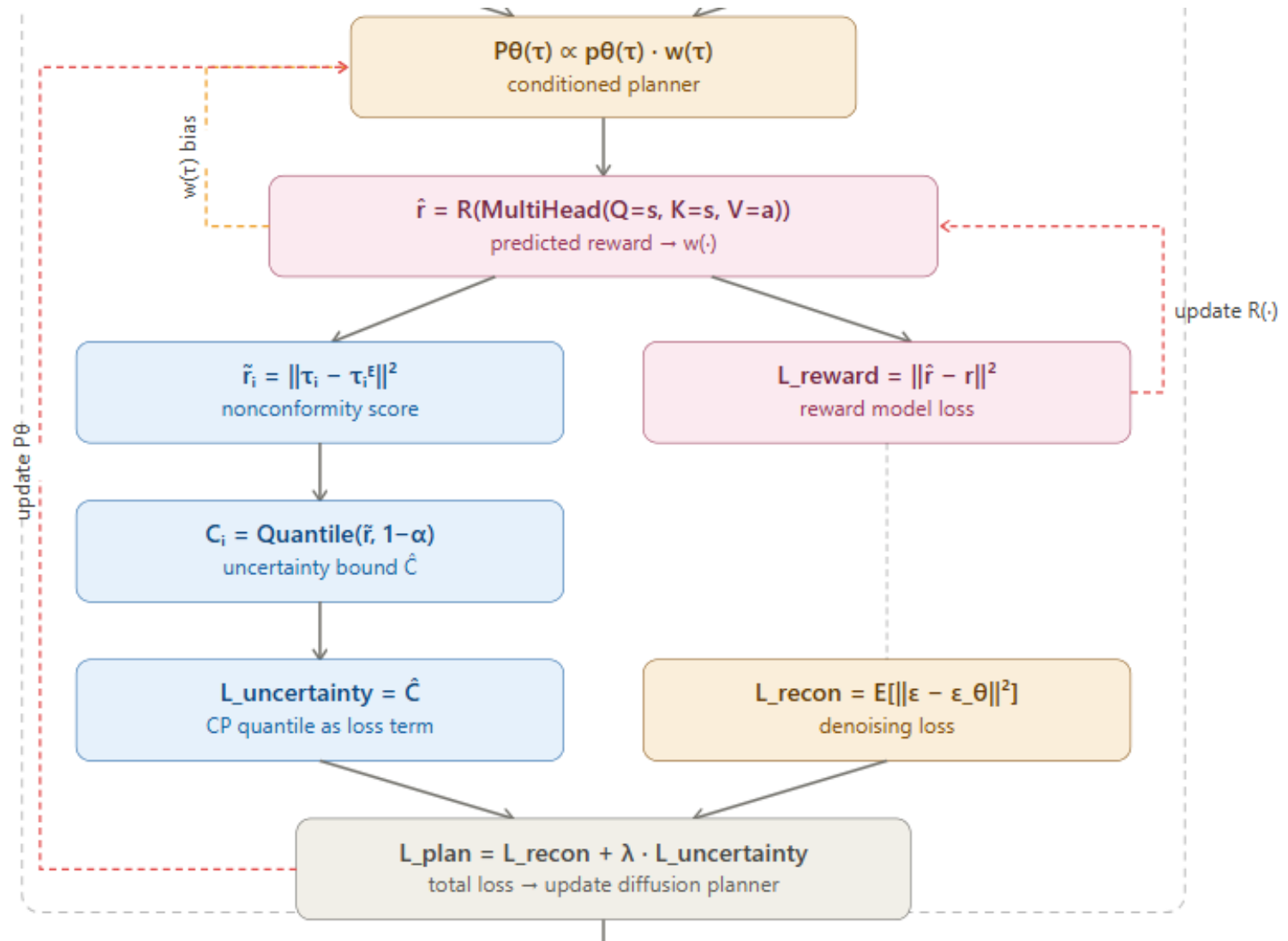
Train to min loss in diffusion planner

- $L_{\text{recon}} = \mathbb{E}[\|\varepsilon - \varepsilon_{\theta}(\tau^n, n)\|^2]$
- diffusion planner: learn to denoise trajectory by predicting added noise ε
- **$L_{\text{plan}} = L_{\text{recon}} + \lambda \cdot L_{\text{uncertainty}}$**
- train to denoise AND close to expert trajectory
→ enforce trajectory τ error $\leq \hat{C}$

train reward model

- $L_{\text{reward}} = \|\hat{r} - r\|^2$
- Fit predicted reward to env reward

- $a_{0:T} = \text{argmin} \sum_t \gamma^t \cdot c(s_t, a_t)$
- RL opt goal: min cumulative cost
- γ uniform discounts



method – 3° Calibration & Test

- **Calibration: Conformal Prediction**

- CP on D_{cal} rewards
- $\hat{q} = [Q^{1-\alpha}, Q^{\alpha}]$
- Two-sided reward prediction interval
 - Narrower $\hat{q}$ = more confident

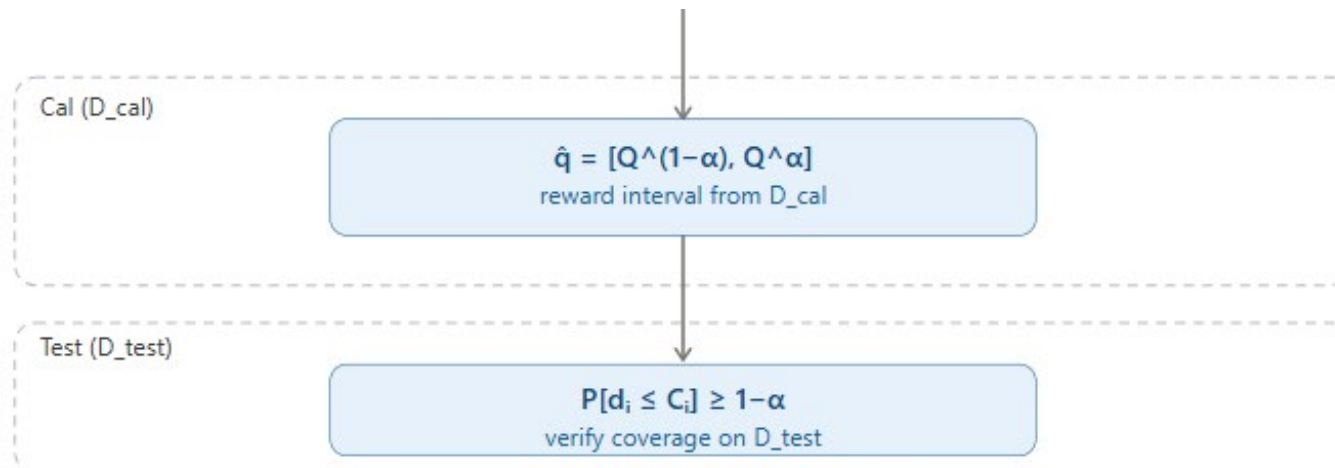
- **Test**

- Verify CP coverage guarantee on D_{test} :
 $P[d_i \leq C_i] \geq 1-\alpha$

- **not make model stronger,
but tells when not to trust it.**

- **CP**

- train to "make model prevent outliers"
- cal to "provide coverage guarantees"



Expectation and logic of the result

- **Diffusers: Blind confidence**
- $L_{\text{recon}} = E[||\varepsilon - \varepsilon_{\theta}(\tau^n, n)||^2]$
- Only learn to denoise, no control on trajectory error
- **PlanCP works better**
- $L_{\text{plan}} = L_{\text{recon}} + \lambda \cdot L_{\text{uncertainty}}$
- $L_{\text{uncertainty}} = \hat{C} = \text{Quantile}(||\tau_i - \tau_i^E||^2, 90\text{th percentile})$
- Far from expert $\tau_i^E \rightarrow$ High nonconformity score $\tilde{r}_i \rightarrow$ High quantile $\rightarrow$ High loss
- quantile constraint penalizes large trajectory errors

Experimental results

– Maze2D

- (a) U-Maze, (c) large: succeeded, and PlanCP's path smoother

- PlanCP higher rewards, narrower reward/uncertainty interval

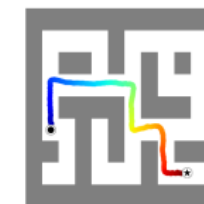
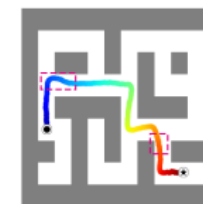
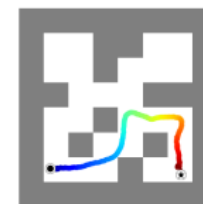
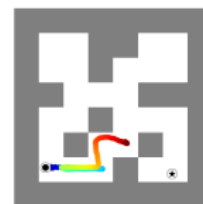
- **Exception: (b) Medium — Diffuser failed**

- PlanCP higher reward
→ CP uncertainty penalty improves planning
- narrowest uncertainty interval
→ Diffuser overconfidence, limited route to explore

- Harder env

→ higher values, wider intervals

Dataset	Diffuser [9]			PlanCP		
	Rewards $\uparrow$	Uncertainty $\downarrow$	Interval $\hat{q} \downarrow$	Rewards $\uparrow$	Uncertainty $\downarrow$	Interval $\hat{q} \downarrow$
Maze2D U-Maze	113.9 ± 3.1	[121.83, 147.19]	25.36	116.4 ± 3.2	[121.11, 142.85]	21.74
Maze2D Medium	121.5 ± 2.7	[132.77, 141.85]	9.08	128.5 ± 2.5	[132.77, 142.99]	10.22
Maze2D Large	123.0 ± 6.4	[92.11, 163.23]	71.12	130.9 ± 5.6	[97.75, 168.46]	70.71
Average	119.5			125.3		



Diffuser

PlanCP

Diffuser (Failed)

PlanCP

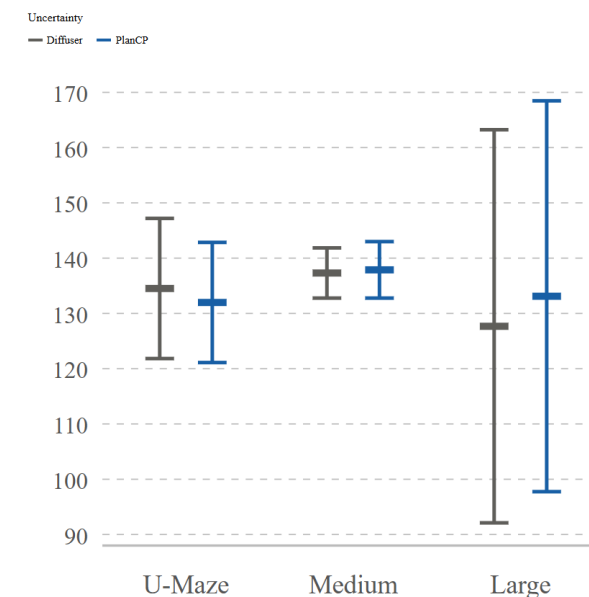
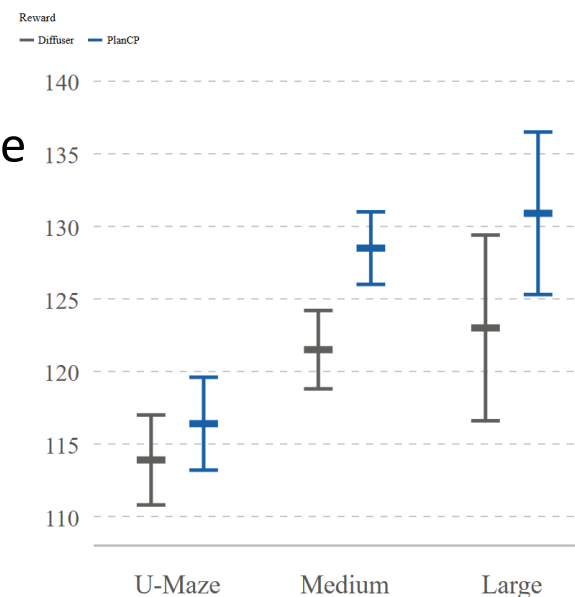
Diffuser

PlanCP

(a) Maze2D U-Maze

(b) Maze2D Medium

(c) Maze2D Large



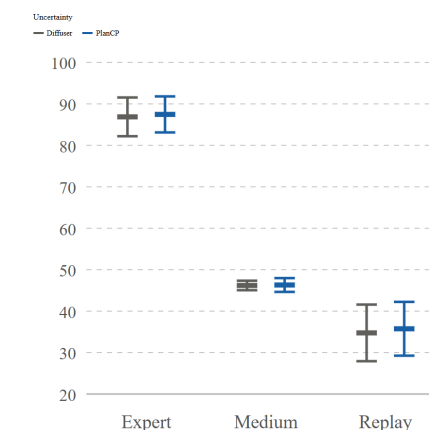
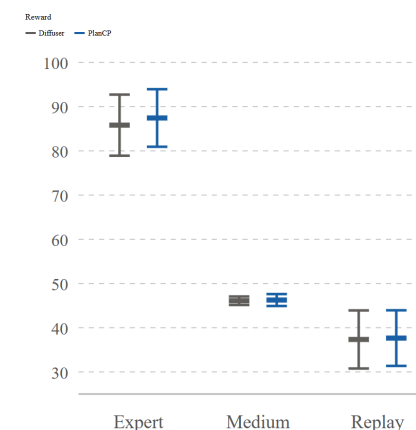
Experimental results

– Offline RL

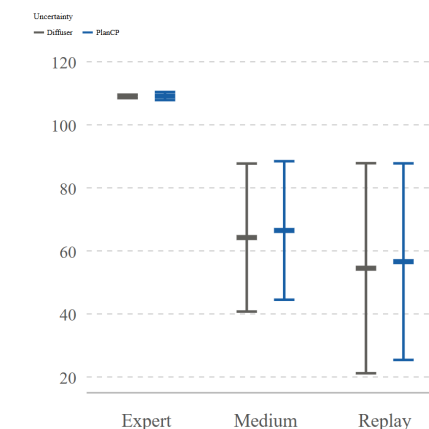
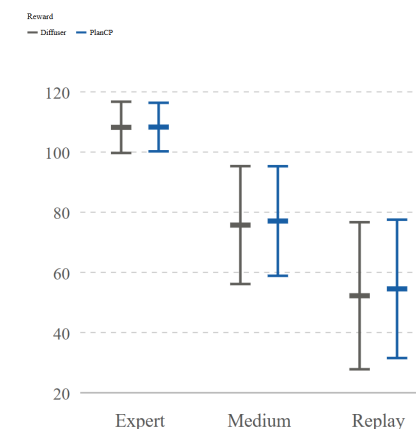
Dataset	Environment	Diffuser [9]			PlanCP		
		Rewards $\uparrow$	Uncertainty $\downarrow$	Interval $\hat{q} \downarrow$	Rewards $\uparrow$	Uncertainty $\downarrow$	Interval $\hat{q} \downarrow$
Medium-Expert	HalfCheetah	85.82 ± 6.91	[82.20, 91.55]	9.34	87.44 ± 6.51	[83.12, 91.81]	8.68
Medium-Expert	Walker2d	108.22 ± 8.52	[108.48, 109.58]	1.10	108.31 ± 8.07	[107.86, 110.43]	2.56
Medium	HalfCheetah	$46.10 \pm \mathbf{0.96}$	[45.05, 47.34]	2.29	46.26 ± 1.35	[44.64, 47.97]	3.33
Medium	Walker2d	75.73 ± 19.59	[40.76, 87.72]	46.96	77.08 ± 18.21	[44.51, 88.46]	43.95
Medium-Replay	HalfCheetah	37.34 ± 6.57	[27.92, 41.58]	13.66	37.65 ± 6.29	[29.25, 42.23]	12.98
Medium-Replay	Walker2d	52.24 ± 24.44	[21.18, 87.83]	66.65	54.53 ± 23.00	[25.41, 87.78]	62.37
Average		67.58			68.55		

- PlanCP higher rewards, narrower reward/uncertainty interval
- **Exceptions have no explanation, but both are not significant**
 - Expert Walker2d, Medium halfcheetah
- env-> reward variation
 - HalfCheetah simpler-> narrow intervals
 - Walker2d Harder→ wide intervals
- Higher data quality
 - higher values, narrower intervals
- **Exception** Medium HalfCheetah: narrowest intervals
 - simple + mid-range quality
 - concentrated predictions

halfcheetah



walker2d



findings and insights

- results: not significant improvement in interval
 - but in maze-medium, diffuser fails
 - PlanCP quantile constraint improves planning
- tech Innovation
 - Differentiable CP quantile loss via differentiable ranking, enable end-to-end backprop
 - CP in training to reduce uncertainty, improve performance

limitation and suggestion

- **No ablation study**
 - can't confirm whether improvement is from CP
 - Alternatively, use other uncertainty constraint
 - (e.g., Bayesian interval, bootstrap, MC dropout; used in **Online RL**).
- **No explanations for all performance**
- **fixed datasets, offline**
 - not fully address OOD
 - CP to penalize high-error trajectories during training
 - implicitly, as learning-based(c.f. planning-based), it may generalize beyond seen data
 - if online, CP cannot apply
 - Tested only in D4RL simulation environment, offline
 - single env, only 3 fixed datasets.
 - not on real robots
- **Lack comparison**
 - only compared with a simple diffuser
 - No comparison with other **planning under uncertainty** literature
 - (e.g. non-parametric models, robust optimization)

conclusion and future work

- Conclusion
 - **PlanCP = diffusion planner + CP quantile as training loss**
 - maintains accuracy, while reduce and quantify uncertainty with coverage guarantee
- future work
 - RL opt goal = $\sum_t \gamma^t \cdot c(s_t, a_t)$, uniform discount γ
 - not accurately reflect actual importance
 - only Transformer attention learns importance in trajectory generation while training
 - Online RL(e.g. autonomous driving, safety-critical applications)
 - cannot apply CP
 - No justification for Diffusion Planner, over RL-guided Diffusion
 - both use Diffusion to generate full trajectory

Method	difference	Trade-off
Diffusion Planner	• Train on expert demos; • Direct trajectory denoising	• Stable, offline • bounded by expert quality
RL-guided Diffusion	• RL gradient fine-tunes the denoising	• Can surpass expert • training is unstable and costly, requires online interaction

Reference of images

- Predicting trajectory destinations via diffusion model
 - <https://www.geeksforgeeks.org/artificial-intelligence/diffusion-models-in-machine-learning/>
 - Q-learning Decision Transformer: Leveraging Dynamic Programming for Conditional Sequence Modelling in Offline RL
 - Gymnasium Documentation
[Rombach & Blattmann, et al. 2022](#)
- Conformal Prediction for Causal Effects of Continuous Treatments